

Junior Developer Take-Home Assessment

Loyalty Program

Time limit: 2 days

1. Overview

Build a simple Loyalty Program web application where users can register, upload purchase receipts, and receive vouchers after their receipts are validated by an administrator.

The assessment evaluates your ability to build a small full-stack application, work with a database and API, implement business logic, and handle authentication, authorization, validation, and maintainable code.

2. Required Technology Stack

- **Frontend:** React
- **Backend:** Node.js + Express
- **Database:** PostgreSQL
- **ORM/query library:** Your choice (Prisma, Sequelize, raw SQL, etc.)
- **Authentication:** Your choice (JWT, session-based, etc.)

You may use additional libraries where appropriate. AI-assisted development tools such as ChatGPT, Claude, Cursor, or GitHub Copilot are allowed. You must understand and be able to explain the code you submit.

3. User Features

3.1 Registration and Login

- Users can register using an email address or phone number.
- OTP verification is not required.
- Users can log in and log out.
- Passwords must not be stored as plain text.

3.2 User Dashboard

Create a simple dashboard showing useful information, such as:

- Welcome message
- Number of pending receipts
- Number of approved receipts
- Number of available vouchers

3.3 Upload Receipt

Users can submit a purchase receipt containing at least:

- Receipt image/file
- Order ID
- Purchase date
- Purchase amount

Every newly submitted receipt must have a status of **PENDING**.

3.4 Receipt History

Users can view their submitted receipts, including:

- Order ID
- Purchase date
- Amount
- Receipt status
- Submission date

A user must only be able to view receipts belonging to their own account.

3.5 Voucher Page

Users can view vouchers issued to them. A voucher should contain enough information to identify the voucher and its source receipt.

3.6 User Settings

Users should be able to update profile information, such as:

- Name
- Email
- Phone number
- Other reasonable profile fields

4. Admin Features

4.1 Admin Authentication

Provide a way for an administrator to access the admin area. Normal users must not be able to access admin-only functionality.

4.2 Admin Dashboard

Create a simple dashboard showing useful statistics, for example:

- Number of pending receipts
- Number of approved receipts
- Number of rejected receipts
- Number of vouchers issued

4.3 Receipt Validation

Create an admin page displaying submitted receipts and allowing an administrator to validate them.

- View receipt details
- Approve a receipt
- Reject a receipt

Receipt status flow:

PENDING	→	APPROVED
PENDING	→	REJECTED

When a receipt is approved, the system must automatically create a voucher for the user. When rejected, the receipt remains rejected and no voucher is created.

5. Business Rules

1. Every newly uploaded receipt starts as PENDING.

2. Only an administrator can approve or reject receipts.
3. Users can only view their own receipts and vouchers.
4. An approved receipt should generate exactly one voucher.
5. A rejected receipt must not generate a voucher.
6. An already approved receipt should not generate another voucher if the approval action is repeated.
7. Important business rules must be enforced by the backend, not only by the frontend.

6. Database

Design the PostgreSQL database yourself. Identify the required tables, relationships, fields, indexes/constraints where appropriate, and statuses. At minimum, the system must represent users, receipts, and vouchers. You may add additional tables if needed.

7. API

Create a REST API for communication between the React frontend and Express backend. Your API should include appropriate endpoints for:

- Authentication
- User profile/settings
- Receipt submission and retrieval
- Voucher retrieval
- Admin receipt validation

Use appropriate HTTP methods, status codes, request validation, and error responses.

8. File Upload

Receipt files must be uploaded and stored so that the application can display or retrieve them later. Local storage is acceptable for this assessment. Consider reasonable file type and file size validation.

9. Error Handling & Validation

Consider and handle cases such as:

- Invalid login credentials
- Duplicate registration
- Missing receipt information
- Invalid amount
- Invalid file type
- Receipt not found
- Unauthorized access
- Attempting to approve an already processed receipt

10. UI / UX

The application does not need a highly polished design. Focus on usability, clear navigation, readable information, and sensible feedback for successful or failed actions.

11. Submission Requirements

- Source code in a Git repository.
- README containing setup and run instructions.

- Database setup/migration/seed instructions.
- Environment variable example file (for example, `.env.example`).
- Brief explanation of application architecture and major design decisions.
- If AI-assisted development was used, include a short note in the README describing how it was used.

12. What We Will Evaluate

- Correctness and completeness of required features
- Code structure and maintainability
- React implementation
- Node.js / Express API design
- PostgreSQL/database design
- Authentication and authorization
- Validation and error handling
- Business logic correctness
- Security awareness
- README and overall project organization

13. Optional Enhancements

These are not required. Complete the core requirements first.

- Receipt image preview
- Pagination
- Search/filter/sort for admin receipts
- Voucher expiry date
- Voucher redemption/usage
- Email notification after receipt approval/rejection
- Automated tests
- Docker setup

14. Important

You are not expected to implement every possible feature. Prioritize the required functionality and make reasonable technical decisions where the requirements are not explicit.

During the follow-up interview, you may be asked to explain your code, database design, API decisions, and how you would handle certain edge cases.